ARM 305 – Cloud, Dev, Edge and Fog Computing

Mini Project

1. Project Objective

Build a locally runnable Python simulator that accepts computing tasks, selects an Edge, Fog, or Cloud node, simulates resource use, and reports performance and security results. Real cloud infrastructure, actual virtual machines, a hypervisor, and a paid cloud account are not required.

2. Required Features

FEATURE	MINIMUM EXPECTATION
Node/resource simulator	At least 3 nodes—one Edge, one Fog, and one Cloud—with CPU, memory, latency, and current utilization.
Workload generator	Create or enter at least 10 tasks. Each task must include CPU and memory requirements plus latency sensitivity (low/medium/high).
Placement algorithm	Implement one rule-based algorithm that selects a node using latency and available capacity. Explain the rules in the report.
Virtualization concept	Maintain simulated VM records with VM ID, host node, allocated CPU/RAM, and status. Support create and stop; no hypervisor is required.
Basic security	Use a simple login/role check or tenant ID. Demonstrate that a tenant cannot view another tenant's protected task records. Do not store real passwords or sensitive data.
QoS and results	Show placement decisions, simulated latency, resource utilization, successful/failed tasks, and total task count. Include a results table and at least one chart.

3. Suggested Technology

Required: Python, pandas or built-in Python data structures, and matplotlib for at least one chart. Choose a command-line menu or a simple Streamlit app. Save results to CSV or JSON. Flask/FastAPI, REST, SOAP, SQLite, Docker, and real cloud services are not required. Include a requirements.txt file for any external Python packages used.

4. Required Experiments

- Run the same workload with at least two placement policies (for example, lowest-latency-first and capacity-aware) and compare the results.
- Run an overload scenario and show whether a task is rejected or assigned to another suitable node.
- Demonstrate an unauthorized tenant-access attempt and show that access is denied.
- Compare results before and after changing a node's capacity or latency.

For each experiment, record the setup, expected outcome, actual outcome, and evidence. Compare placement outcomes, resource utilization, and simulated QoS where applicable.

5. Expected Deliverables and Demonstration

DELIVERABLE	MINIMUM EXPECTATION
Architecture	One diagram showing users/tasks, placement logic, Edge/Fog/Cloud nodes, and results.
Working code	Runnable Python source code, a requirements.txt file, and setup/execution instructions in README.
Experiments	At least 3 test scenarios, including overload and an unauthorized-access attempt; include the required policy and capacity/latency comparisons.
Results	A results table and at least one chart or clear visualization showing placement, resource utilization, and QoS measures.
Project report	A concise 5–8 page report explaining the objective, architecture, algorithm, implementation, experiments, results, and limitations.
Presentation	A 5–7 slide presentation summarizing the system, algorithm, experiments, results, and limitations.
Demonstration	A short live demonstration: run the application, submit sample tasks, show placement decisions, and explain QoS/security outputs.

6. Evaluation Rubric

EVALUATION COMPONENT	MARKS
Problem statement and architecture diagram	10
Node/resource and workload simulation	20
Edge/Fog/Cloud placement algorithm	20
VM simulation and basic access control	15
QoS calculations, experiments and chart	15
Testing, code quality and README/report	10
Demonstration and individual viva	10
TOTAL	**100**

7. Final Submission

- **Single ZIP file:** Submit one ZIP file per team. Only one student from the team should submit it.
- **Late submission penalty:** 10 points will be deducted for every 6 hours of delay.
- **ZIP contents:** runnable source code, requirements.txt, README/setup instructions, architecture diagram, 5–8 page report, experiment/test evidence and screenshots, results table/chart, and 5–7 slide presentation.
- **Individual viva:** Every student should be able to explain their contribution and answer questions about the complete simulator.

IMPORTANT: Ensure the ZIP contains all listed deliverables and that the project runs by following the README instructions. REST/SOAP API documentation, real cloud deployment, Docker, and actual VMs are not part of this mini-project.